Thomas Etienne (8h)

Jake MacCallum (10h)

Compte rendu du TP Igel Ärgern

Introduction

Le but du projet est d'implémenter une course entre herissons. Il s'agit d'un jeu tour à tour dans la console. Ce projet nous a permis de nous familliariser avec le C, d'apprendre à collaborer et à travailler en équipe, d'organiser nos idées en C dans un projet plus large.

Construire et lancer le projet

Ce projet a été réalisé à l'aide de xmake, qui permet de construire, et lancer le projet, ainsi que de configurer clangd :

- configurer clangd

```
xmake project -k compile_commands
```

- changer de mode compilation

```
xmake f -m debug
```

Le projet dipose de 3 profiles de compilations, `release` (optimisation activées), `debug` (symboles de debugage et asan) et `releasedbg` (optimisation et symboles de debugage).

- Générer le projet et lancer le projet

```
xmake run
```

Il est possible de seulement générer les binaires via la commande `xmake build`, qui apparaîtront dans un sous dossier du dossier build (en fonction de l'architecture et de la plateforme utilisée).

Pour faciliter la construction du projet, un `makefile` qui construit le jeu et les tests, correspondant au profil de debugage est fourni. pour regenerer le makefile avec un autre profil (par exemple en releasedbg)

```
xmake f -m releasedbg
xmake project -k make
```

Test

Le projet comprend aussi une série de tests, réuni dans le fichier `src/test.c` qui peut être lancé avec la commande

```
xmake run test
```

Conventions de nommages

- Les noms de structures sont en PascalCase
- Les noms de fonctions sont en snake_case, si elles sont rattachées a une structure, le nom de celle-ci est utilisée comme préfixe
- Les constantes et `#define` sont en UPPERCASE.

Synthèse

L'implémentation du jeu essaye de séparer au maximum la représentation du jeu, le rendu et les interactions avec l'utilisateur, respectivements dans les fichiers `game.h/game.c`, `canvas.h/canvas.c` et `console_application.h/console_application.c`.

- Le jeu est représenté par un tableau a deux dimension de cellules dans lequels sont stockés les hérissons. Chaque cellule contient une pile dans laquelle on identifie chaque hérisson par le joueur auquel il appartient.
- Le rendu du jeu est réalisé dans un tampon avant d'être envoyé dans la sortie standard, cela permet de simplifier le code du rendu à l'aide de rendu le ligne
- La plupart des allocations sont dynamiques pour pouvoir configurer le jeu sans le recompiler. Pour moddifer le nombre de joueurs, de hérissons ou la taille de la grille, il suffit de de passer des arguments supplémentaires. Si ce ci ne sont pas correctement reconnus, le programme affiche un message d'aide. Le programme vérifie que ces entrées sont correctement formées et sont des valeurs valide .

Beaucoup de temps à été passé à unifier deux conventions sur l'accés aux cellules, la première que l'on a gardé voit le plateau comme un plan avec son origine en haut à gauche et la deuxième le voit comme un tableau de lignes et de colonnes L'une est plus claire pour le rendu, l'autre plus claire pour le déroulement et la logique du jeu. On a préféré garder la même dans l'intégrité du code final.

Une extension intéressante pourrait être des déplacements aléatoires de cases piège, peut-être même en déplaçant les hérissons qui s'y trouvent.

Outils utilisés lors du developpement

Les sources sont **strictement** conforme à la norme C11 et compilent sous Linux et Windows avec les drapeaux `-Wall -Wextra -Werror`. Par conséquent, aucune extension de compilateur non standard comme `#pragama once` n'a été utilisé.

- éditeur de texte : Zed
- compilateurs : Gcc, MSVC
- build system : xmake
- LSP : Clangd
- debugger: LLDB

Bibliographie

- la documentation de C
- la documentation de Xmake